
Laboratoire 6

Mesure de consommation énergétique

Départements : TIC
Unité d'enseignement HPC

Auteur : Urs Behrmann
Professeur : Alberto Dassatti
Assistant : Bruno Da Rocha Carvalho
Classe : HPC
Salle de labo : A09
Date : 19 mai 2026

1 Tâche 1 : SIMD vs séquentiel

1.1 Programme

Code du labo 4 dans `lab06/build/simd_bench`, $n = 30 \times 10^6$, modes `seq` et `simd`. Même checksum.

1.2 Mesures (sudo perf stat)

```
sudo perf stat -e power/energy-pkg/,power_core/energy-core/ \
./simd_bench seq
sudo perf stat -e power/energy-pkg/,power_core/energy-core/ \
./simd_bench simd
```

Version	energy-pkg (J)	energy-core (J)	Temps (s)
séquentielle	6,18	2,76	0,368
SIMD (SoA)	10,39	4,57	0,597

1.3 Interprétation

La version SIMD consomme environ **68 %** d'énergie package en plus (10,39 J vs 6,18 J) pour le même résultat, et prend **62 %** de temps en plus. C'est cohérent : sur mon portable, le SIMD n'accélère pas le calcul (conversion AoS→SoA, boucle simple) mais fait tourner le CPU plus longtemps, donc plus de joules. L'énergie suit surtout le temps actif du package.

2 Tâche 2 : projet ECG (labo 1)

2.1 Outils

- **Global** : `sudo perf stat -e power/energy-pkg/,power_core/energy-core/ ./ecg_energy ...`
- **Ciblé** : `PerfManager + sudo ../scripts/run_ecg_perf_control.sh`

Fichier : `lab06/data/test_ecg.csv` (7500 échantillons, 96 pics R).

2.2 Mesure globale

```
sudo perf stat -e power/energy-pkg/,power_core/energy-core/ \
./ecg_energy ../data/test_ecg.csv /tmp/out_global.json
```

Compteur	Énergie / durée
power/energy-pkg/	0,20 J
power_core/energy-core/	0,12 J
Temps perf	0,008 s
ecg_analyze (chrono)	0,334 ms

Le programme lit tout le CSV, analyse la dérivation II, écrit le JSON. L'énergie inclut donc l'I/O et pas seulement le traitement du signal.

2.3 Mesure ciblée (ecg_analyze seulement)

Script avec FIFO et `perf stat -delay=-1 -control`. Résultat :

Compteur	Énergie
power/energy-pkg/	0,02 J
power_core/energy-core/	0,01 J
Temps perf	0,008 s
Temps ecg_analyze (chrono)	0,317 ms

La zone mesurée est très courte (< 1 ms), donc les joules sont petits et un peu bruités.

2.4 Global vs ciblé

Mesure	energy-pkg (J)	energy-core (J)
globale (tout le programme)	0,20	0,12
ciblée (ecg_analyze)	0,02	0,01

Environ **90 %** de l'énergie package du run complet vient du hors-ecg_analyze (parsing CSV des 12 dérivations, écriture JSON). Le traitement du signal reste important en CPU utile, mais sur ce fichier test l'I/O domine la facture énergétique globale.

2.5 Partie coûteuse

Pour réduire la consommation sur cible embarquée, il faudrait viser le format d'entrée (binaire / une seule dérivation) avant d'optimiser encore `ecg_analyze`.

2.6 Amélioration proposée

Réutiliser la mémoire de travail dans `ECG_Context` au lieu de :

- allouer `threshold`, `pref_sum`, `pref_sq` à chaque `ecg_analyze` ;

- utiliser un tableau automatique `double s[n_samples]` sur la pile.

But : moins d'appels à `malloc/free` et moins de trafic mémoire → moins de CPU actif → moins de joules sur une analyse répétée (typique en embarqué).

Code **avant** : `lab06/baseline/` et binaire `ecg_repeat_avant`.

Code **après** : `lab01/src/ecg_processing.c` et `ecg_repeat_apres`.

2.7 Mesure après modification

2000 fois `ecg_analyze` sur le même CSV (96 pics R à chaque fois), hors écriture JSON :

```
cd lab06/build && cmake .. && make ecg_repeat_avant ecg_repeat_apres  
sudo ../scripts/compare_ecg_opt.sh ../data/test_ecg.csv 2000
```

Mesure `perf stat` sur tout le binaire (1 lecture CSV + 2000 analyses). Variante ciblée FIFO : `run_ecg_repeat_perf.sh avant|apres`.

2.8 Comparaison avant / après

`sudo ../scripts/compare_ecg_opt.sh` (2000 analyses, même CSV, 96 pics R) :

Version	Temps réel (s)	energy-pkg (J)	energy-core (J)
avant (malloc + VLA)	0,546	10,22	5,81
après (tampons contexte)	0,500	9,07	5,20
Gain	≈ 8,4 %	≈ 11,3 %	≈ 10,5 %

Moins de `malloc/free` et moins de pression sur la pile ⇒ moins de temps CPU et environ **11 %** de joules en moins sur le package pour 2000 analyses. Sur un run unique `ecg_energy` (0,20 J total), l'écart est noyé par le CSV ; d'où `ecg_repeat_*`.

3 Conclusion

L'idée centrale du labo : **moins le CPU travaille longtemps, moins le package consomme de joules**. Les compteurs RAPL (`power/energy-pkg/`, `power_core/energy-core/`) suivent surtout le temps où le processeur est actif.

SIMD : la version vectorisée est plus lente (0,60 s vs 0,37 s) et consomme plus (10,39 J vs 6,18 J). Pas de gain perf ⇒ pas de gain énergétique — au contraire.

ECG : en répétant `ecg_analyze`, moins de temps (0,50 s vs 0,55 s) va avec moins d'énergie (9,07 J vs 10,22 J, soit ≈ 11 %). Optimiser le code pour finir plus vite, c'est directement réduire la facture énergétique.

Embarqué : viser d'abord ce qui allonge le CPU (I/O, allocations inutiles), puis mesurer les joules. Le temps reste un bon indicateur : s'il baisse, l'énergie baisse en général aussi.